Cloudworx Frontend Architecture

Frontend assessment — short design and prototype report

Date: 17 October 2026 **Prototype:** React, TypeScript, Vite, Three.js, React Three Fiber

1. Overview

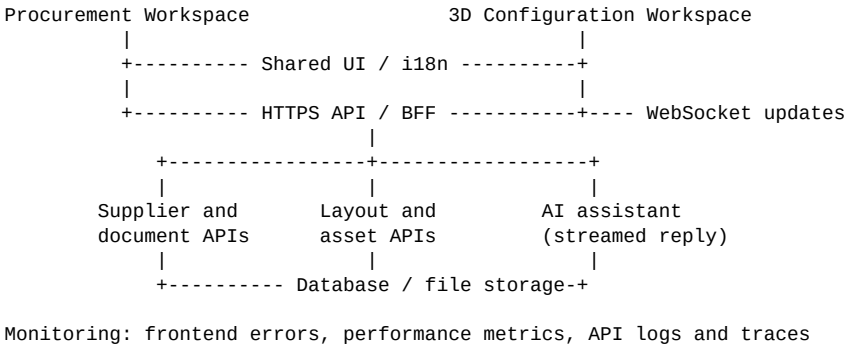
The assessment describes two product areas:

- **Procurement Workspace:** supplier and document tables, RFQs, forms, and an AI assistant.
- **3D Configuration Workspace:** place equipment in a room, check layout rules, and review a bill of materials (BOM).

I would build them as **separate applications** that share a versioned design system, localization helpers, authentication utilities, and telemetry. This lets each product release independently while keeping common controls consistent.

This repository is a small, local 3D-editor prototype. The production services below are a proposed design, not implemented features.

2. Proposed system at a glance



The frontend talks to backend services through an API gateway or Backend-for-Frontend (BFF). HTTPS handles normal requests; WebSockets carry live collaboration or status updates. Large model files are delivered from object storage/CDN rather than bundled into the JavaScript application.

3. State and data flow

For the editor, I would keep one typed layout model as the source of truth. The 3D scene, item list, properties panel, warnings, and BOM are all derived from that model. This avoids the scene showing a different position from the list or BOM.

In this prototype, React `useReducer` manages add, select, move, rotate, delete, undo, and redo. The reducer is small and testable without rendering Three.js. For a larger application, I would keep layout rules in a shared domain package and add server-state caching only where API data needs it.

For server changes, show a clear saving state, retry safe failures, and avoid overwriting newer edits with stale responses. With an unreliable VPN, keep a local draft and reconcile it with the server when the connection returns. Multi-user conflict rules must be agreed with the backend team; I would not silently overwrite another user's update.

4. Large tables and forms

The procurement workspace may show 50,000 suppliers and 200,000 documents. I would not render every row at once. Use server-side filtering, sorting and pagination, plus row virtualization for the visible window. Keep filters and sort order in the URL so a user can return to the same view. Forms should validate near the field, preserve drafts, and clearly explain save failures.

5. 3D model performance

For heavy CAD assets, I would create web-ready models before delivery:

- Remove hidden or unnecessary geometry and reduce polygon count.
- Provide levels of detail (LOD) so distant objects use simpler models.
- Compress geometry and textures where supported.
- Load models on demand and cache reusable assets.
- Reuse repeated geometry/materials and use instancing for repeated items where practical.
- Measure load time, frame rate, draw calls, and memory on the target low-end machine.

The assessment's performance figures should be treated as targets until measured on the agreed hardware. This prototype uses simple box geometry; it does **not** demonstrate a production CAD pipeline or prove a 50 FPS result.

For large layouts, the current proximity checker compares every pair of items. That is easy to understand for a small demo, but it scales poorly as item counts grow. For thousands of items, I would use a spatial grid or another broad-phase index to find nearby candidates, then run the exact clearance check only on those candidates.

6. Embedding and security

If the editor is hosted inside another application, I would define a small, versioned message contract for events such as `ready`, `selectionChanged`, `saveRequested`, and `layoutUpdated`. Both sides must validate the sender's origin and the message shape. Do not put long-lived access tokens in URL parameters or trust arbitrary messages from the host page. Use a short-lived, scoped session established through the approved authentication flow.

If the host blocks iframes, provide a standalone editor route or agree on a supported microfrontend integration. The current prototype has no host integration or token handling.

7. AI assistant workflow

The assistant should **propose** a change, not silently mutate a live layout. A safe flow is:

1. User asks to move a machine or add equipment.
2. The assistant returns a structured proposed change.
3. The UI validates the proposal and displays a preview.
4. The user accepts or rejects it.
5. An accepted change goes through the same layout reducer and validation rules as a manual edit, so it can be undone.

Use a streamed response for a responsive interface, validate all returned fields, and keep authorization and business rules on the server. This is a proposed design only; AI integration is not implemented here.

8. Localization, accessibility and reliability

The production product should support English, Vietnamese, and Simplified Chinese, with currency and number formatting based on locale. Avoid hard-coded user-facing strings and allow longer translations in the layout. The prototype currently supports English and Vietnamese only.

Use semantic HTML, visible keyboard focus, labelled controls, keyboard operation for important actions, and screen-reader announcements for validation or save status. Test with keyboard-only navigation and assistive technology; do not assume a 3D canvas alone is accessible. The current prototype has not had a formal WCAG 2.2 AA audit.

I would add unit tests for layout rules and reducer history, component tests for panels and localization, and end-to-end tests for add/move/rotate/delete/undo/BOM workflows. CI should run type checking, linting, tests and a production build. Production monitoring should capture frontend errors and real-user performance, without logging secrets or sensitive document content.

9. Example workflow

A user adds a cabinet, moves it onto the half-metre grid, and selects it. The reducer updates the layout; the 3D scene, list, properties, warning list and BOM render from the same state. If another item's centre is within 0.6 m, the prototype flags the pair. In production, that simplified centre-distance rule would be replaced or confirmed against the actual clearance requirement. If the user asks the AI assistant to rearrange equipment, the assistant would return a preview for approval rather than applying changes directly.

10. What this repository implements

Implemented in the prototype:

- 20 m × 12 m room and 0.5 m grid.
- Three equipment types using simple 3D boxes.
- Add, select, move with controls or numeric coordinates, rotate and delete.
- Grid snapping and room-boundary clamping.
- Undo/redo, synchronized item list and properties panel, and BOM counts.
- Centre-to-centre proximity warnings at 0.6 m.
- English and Vietnamese labels.

Not implemented or not verified:

- Drag-and-drop placement/direct dragging in the 3D scene.
- True edge-to-edge clearance calculations.
- Simplified Chinese, backend persistence, autosave, collaboration, AI integration, and secure host embedding.
- A production CAD asset pipeline and a measured 500-item/50 FPS benchmark.
- Automated test coverage and a formal accessibility audit.

The prototype's proximity rule is deliberately simple: it measures horizontal centre-to-centre distance, ignoring object footprints. It should not be described as industrial clearance validation.

11. Key decisions and trade-offs

- **One layout state:** keeps every panel consistent; derived values are recalculated from the same items.
- `useReducer` **for the prototype:** enough for this small local workflow and makes edits explicit. A larger product can move the domain logic into a shared package without starting with a heavy state library.
- **Separate application release trains:** lets procurement changes and 3D-editor changes ship independently.
- **Measure before optimizing:** performance targets need a named machine, representative assets and repeatable measurements.
- **Human approval for AI edits:** preview, validation and undo reduce the risk of unexpected layout changes.

12. Source map

- `src/App.tsx` — connects panels and derives selected item, warnings and BOM.
- `src/features/layout/layoutTypes.ts` — equipment types, room dimensions, snapping and bounds.
- `src/features/layout/layoutReducer.ts` — layout actions and undo/redo history.
- `src/features/layout/layoutRules.ts` — current centre-distance warning rule.
- `src/components/SceneCanvas.tsx` — Three.js scene and equipment rendering.
- `src/components/EquipmentPalette.tsx`, `PropertiesPanel.tsx`, `ItemList.tsx`, `BillOfMaterials.tsx`, `Toolbar.tsx` — user interface panels.
- `src/i18n.ts` — current English and Vietnamese labels.

Assumptions: backend APIs, identity provider, hosting rules, CAD conversion pipeline, and exact clearance requirements were not supplied. The production design above is therefore a proposal to validate with the product, backend, security, and 3D teams.